

Código

Clases

Gestion:

```
public class Gestion {  
  
    public static void main(String[] args) {  
        java.awt.EventQueue.invokeLater(new Runnable() {  
            public void run() {  
                new VentanaPrincipal().setVisible(true);  
            }  
        });  
    }  
}
```

Tarea:

```
public class Tarea {  
    private static int numero = 0;  
    private final int codigo;  
    private String titulo;  
    private String prioridad;  
    private String modulo;  
    private String fechaCreacion;  
    private String fechaEntrega;  
    private String estado;  
  
    public Tarea() {  
        incNumero();  
        this.codigo = numero;  
    }  
  
    private static void incNumero() {  
        numero++;  
    }  
  
    public int getCodigo() {  
        return codigo;  
    }  
  
    public String getTitulo() {  
        return titulo;  
    }  
}
```

```
public void setTitulo(String titulo) {
    this.titulo = titulo;
}

public String getPrioridad() {
    return prioridad;
}

public void setPrioridad(String prioridad) {
    this.prioridad = prioridad;
}

public String getModulo() {
    return modulo;
}

public void setModulo(String modulo) {
    this.modulo = modulo;
}

public String getFechaCreacion() {
    return fechaCreacion;
}

public void setFechaCreacion(String fechaCreacion) {
    this.fechaCreacion = fechaCreacion;
}

public String getFechaEntrega() {
    return fechaEntrega;
}

public void setFechaEntrega(String fechaEntrega) {
    this.fechaEntrega = fechaEntrega;
}

public String getEstado() {
    return estado;
}

public void setEstado(String estado) {
    this.estado = estado;
}

@Override
public String toString() {
```

```

        return "Tarea{" + "codigo=" + codigo + ", titulo=" + titulo + ", prioridad=" + prioridad + ",
modulo=" + modulo + ", fechaCreacion=" + fechaCreacion + ", fechaEntrega=" +
fechaEntrega + ", estado=" + estado + "}";
    }
}

```

Ventana Principal

```

import java.awt.event.ActionEvent;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import javax.swing.JFormattedTextField;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import java.text.Collator;
import java.util.Locale;

public class VentanaPrincipal extends javax.swing.JFrame {

    private static final java.util.logging.Logger logger =
java.util.logging.Logger.getLogger(VentanaPrincipal.class.getName());
    public VentanaPrincipal() {
        initComponents();

        btnActualizar.addActionListener(this::actualizarTarea);
        jButtonOrdenar.addActionListener(this:: jButtonOrdenarActionPerformed);
        jButtonOrdenarModulo.addActionListener(this::
jButtonOrdenarModuloActionPerformed);
        jButtonOrdenarFechaEntr.addActionListener(this::
jButtonOrdenarFechaEntrActionPerformed);
        // Inicializo el arrayList
        listaTareas = new ArrayList<>();
        listaTareasTerminadas = new ArrayList<>();
        // Esto evita que se borre el texto si hay error
        txtFechaEntrega.setFocusLostBehavior(JFormattedTextField.COMMIT_OR_REVERT);

        txtFechaCreacion.setText(
            LocalDate.now().format(DateTimeFormatter.ofPattern("d MMM y"))
        );
    }
    @SuppressWarnings("unchecked")

    private void btnGuardarActionPerformed(java.awt.event.ActionEvent evt) {

        try {
            DateTimeFormatter formato = DateTimeFormatter.ofPattern("d MMM y");

```

```

String textoFecha = txtFechaEntrega.getText();

if (textoFecha.trim().isEmpty()) {
    throw new Exception("La fecha de entrega no puede estar vacía.");
}
LocalDate fechaValida = LocalDate.parse(textoFecha, formato);

Tarea t = new Tarea();

t.setTitulo(txtTitulo.getText());
t.setPrioridad(comboPrioridad.getSelectedItem().toString());
t.setModulo(comboModulo.getSelectedItem().toString());
t.setFechaCreacion(txtFechaCreacion.getText());
t.setFechaEntrega(txtFechaEntrega.getText());
t.setEstado("pendiente");
System.out.println("Tarea Creada Correctamente: " + t.getCodigo());
System.out.println(t);
listaTareas.add(t);

boolean tablaActualizada = actualizarTabla();

if(tablaActualizada){
    JOptionPane.showMessageDialog(this,
        "Tarea agregada correctamente");
}

limpiarCampos();

} catch (Exception e) {
    javax.swing.JOptionPane.showMessageDialog(this,
        "Formato de fecha incorrecto. Debe ser: 18 abr 2026",
        "Error de Validación",
        javax.swing.JOptionPane.ERROR_MESSAGE);
    txtFechaEntrega.requestFocus();
    txtFechaEntrega.selectAll();
}
}

private void btnBuscarActionPerformed(java.awt.event.ActionEvent evt) {

    try {
        int codigoBuscado = Integer.parseInt(txtCodigo.getText());

        tareaSeleccionada = null;

        for (Tarea t : listaTareas) {
            if (t.getCodigo() == codigoBuscado) {
                tareaSeleccionada = t;
            }
        }
    }
}

```

```

        if (t.getEstado().equalsIgnoreCase("resuelta")) {
            chkResuelta.setSelected(true);
        } else {
            chkResuelta.setSelected(false);
        }

        javax.swing.JOptionPane.showMessageDialog(this, "Tarea encontrada");

        return;
    }

}

javax.swing.JOptionPane.showMessageDialog(this, "Código no existe!");

txtCodigo.setText("");

} catch (Exception e) {

    javax.swing.JOptionPane.showMessageDialog(this, "Código inválido!");
}

}

private void jButtonOrdenarActionPerformed(java.awt.event.ActionEvent evt) {
    if(listaTareas == null){
        JOptionPane.showMessageDialog(this,
            "No hay tareas creadas");
    }
    else{

        for(int i = 0; i< listaTareas.size()-1;i++){
            for(int j = i+1; j<listaTareas.size(); j++){
                int prioridadTarea1 = 0;
                int prioridadTarea2 = 0;

                Tarea t1 = listaTareas.get(i);
                Tarea t2 = listaTareas.get(j);

                String tarea1 = t1.getPrioridad();
                prioridadTarea1 = nivelDePrioridad(tarea1);

                String tarea2 = t2.getPrioridad();
                prioridadTarea2 = nivelDePrioridad(tarea2);

                if(prioridadTarea1 < prioridadTarea2 ){
                    listaTareas.set(i, t2);
                    listaTareas.set(j,t1);
                }
            }
        }
    }
}

```

```

        }
    }
}
actualizarTabla();
}
}

private void jButtonOrdenarModuloActionPerformed(java.awt.event.ActionEvent evt) {
    String modulo = null;
    Collator collator = Collator.getInstance(new Locale("es","ES"));
    listaTareas.sort((t1,t2) -> //Uso de lamda para crear ka función de manera rápida
        collator.compare(t1.getModulo(), t2.getModulo()));
    actualizarTabla();
}

private void jButtonOrdenarFechaEntrActionPerformed(java.awt.event.ActionEvent evt) {
    for(int i = 0; i < listaTareas.size() -1 ; i++){
        for(int j = i+1; j < listaTareas.size(); j++){
            Tarea t1 = listaTareas.get(i);
            Tarea t2 = listaTareas.get(j);

            String fechaEntrega1 = t1.getFechaEntrega();
            String fechaEntrga2 = t2.getFechaEntrega();

            DateTimeFormatter formatter =
                DateTimeFormatter.ofPattern("d MMM y", new Locale("es", "ES"));

            LocalDate fecha1 = LocalDate.parse(fechaEntrega1, formatter);
            LocalDate fecha2 = LocalDate.parse(fechaEntrga2, formatter);
            if(fecha2.isBefore(fecha1)){
                listaTareas.set(j, t1);
                listaTareas.set(i, t2);
            }
        }
    }
    actualizarTabla();
}

private boolean actualizarTabla() {
    DefaultTableModel modelo = (DefaultTableModel) jTable1.getModel();

    modelo.setRowCount(0);

    for (Tarea t : listaTareas) {
        if (t.getEstado().equalsIgnoreCase("pendiente")) {
            Object[] fila = {
                t.getCodigo(),
                t.getTitulo(),
                t.getPrioridad(),
                t.getModulo(),
            }
        }
    }
}

```

```

        t.getFechaCreacion(),
        t.getFechaEntrega(),
        t.getEstado()
    };

    modelo.addRow(fila);
    }
}
return true;
}

private void limpiarCampos() {

    txtTitulo.setText("");
    comboPrioridad.setSelectedIndex(0);
    comboModulo.setSelectedIndex(0);
    txtFechaEntrega.setText("");
    txtFechaCreacion.setText(
        LocalDate.now().format(DateTimeFormatter.ofPattern("d MMM y"))
    );
    txtTitulo.requestFocus();
}

private void actualizarTarea(java.awt.event.ActionEvent evt) {

    if (tareaSeleccionada == null) {
        JOptionPane.showMessageDialog(this,
            "Primero debes buscar una tarea");
        return;
    }

    if (!chkResuelta.isSelected()) {
        JOptionPane.showMessageDialog(this,
            "Marca la tarea como resuelta para borrarla");
        return;
    }

    int option = JOptionPane.showConfirmDialog(
        this,
        "¿Seguro que quieres borrar la tarea?",
        "Confirmar",
        JOptionPane.YES_NO_OPTION
    );

    if (option != JOptionPane.YES_OPTION) {
        return;
    }
}

```

```

Tarea t1 = tareaSeleccionada;
listaTareas.remove(tareaSeleccionada);
String pendiente = "Finalizado";
t1.setEstado(pendiente);
listaTareasTerminadas.add(t1);

tareaSeleccionada = null;
txtCodigo.setText("");
chkResuelta.setSelected(false);

boolean tablaActualizada = actualizarTabla();

if(tablaActualizada){
    JOptionPane.showMessageDialog(this,
        "Tarea eliminada correctamente");
}
actualizarTablaTareaFinalizada();
}

private int nivelDePrioridad(String prioridadTarea){
    int prioridadTareaNum = 0;
    switch(prioridadTarea){
        case "Alta":
            prioridadTareaNum = 3;
            break;
        case "Media":
            prioridadTareaNum = 2;
            break;
        case "Baja":
            prioridadTareaNum = 1;
            break;
    }
    return prioridadTareaNum;
}

private void actualizarTablaTareaFinalizada() {
    DefaultTableModel modelo = (DefaultTableModel) jTableTareaFinalizada.getModel();

    modelo.setRowCount(0);
    for (Tarea t : listaTareasTerminadas) {
        Object[] fila = {
            t.getCodigo(),
            t.getTitulo(),
            t.getPrioridad(),
            t.getModulo(),
            t.getFechaCreacion(),
            t.getFechaEntrega(),
        }
    }
}

```



```

        t.getEstado()
    };

    modelo.addRow(fila);
}
}

private ArrayList<Tarea> listaTareas = null;
private Tarea tareaSeleccionada = null;
private ArrayList<Tarea> listaTareasTerminadas = null;
}

```

Pruebas de ejecución:

Crear Tarea

Título:

Prioridad:

Módulo:

Fecha creación:

Fecha entrega:

Actualizar Tarea

Código:

☐ Resuelta

Tareas Pendientes

Código	Título	Prioridad	Módulo	Fecha creación	Fecha entrega	Estado

Tareas Finalizadas

Código	Título	Prioridad	Módulo	Fecha creación	Fecha entrega	Estado

Crear Tarea

Título: Interfaces

Prioridad: Alta

Módulo: Programación

Fecha creación: 5 may 2026

Fecha entrega: 7 may 2026

Guardar

Actualizar Tarea

Código:

Buscar

☐ Resuelta

Actualizar

Tareas Pendientes

Ordenar por Prioridad

Ordenar por Modulo

Ordenar por Fecha de entrega

Código	Título	Prioridad	Módulo	Fecha creación	Fecha entrega	Estado
1	Interfaces	Alta	Programación	5 may 2026	7 may 2026	pendiente

Mensaje

Tarea agregada correctamente

Aceptar

Tareas Finalizadas

Código	Título	Prioridad	Módulo	Fecha creación	Fecha entrega	Estado

Actualizar Tarea

Código: 1

Buscar

☐ Resuelta

Actualizar

Fecha de entrega

Prioridad	Módulo	Fecha creación	Fecha entrega	Estado
	Programación	5 may 2026	10 may 2026	pendiente

Mensaje

Tarea encontrada

Aceptar

Actualizar Tarea

Código:

Buscar

☒ Resuelta

Actualizar

por Fecha de entrega

Prioridad	Módulo	Fecha creación	Fecha entrega	Estado
	Programación	5 may 2026	10 may 2026	pendiente

Confirmar

×



¿ Seguro que quieres borrar la tarea?

Prioridad	Módulo	Fecha creación	Fecha entrega	Estado
-----------	--------	----------------	---------------	--------



Mensaje

Tarea eliminada correctamente

Aceptar

Tareas Finalizadas						
Código	Título	Prioridad	Módulo	Fecha creación	Fecha entrega	Estado
1	Interfaces	Alta	Programación	5 may 2026	10 may 2026	Finalizado

Ordenado por Prioridad:

Tareas Pendientes

Ordenar por Prioridad	Ordenar por Modulo	Ordenar por Fecha de entrega				
Código	Título	Prioridad	Módulo	Fecha creación	Fecha entrega	Estado
3	Prueba 2	Alta	Bases de datos	5 may 2026	19 ene 2026	pendiente
4	Prueba 3	Media	Programación	5 may 2026	21 jun 2026	pendiente
2	Prueba 1	Baja	Programación	5 may 2026	6 may 2026	pendiente

Ordenado por Modulo:

Tareas Pendientes						
Ordenar por Prioridad Ordenar por Modulo Ordenar por Fecha de entrega						
Código	Título	Prioridad	Módulo	Fecha creación	Fecha entrega	Estado
3	Prueba 2	Alta	Bases de datos	5 may 2026	19 ene 2026	pendiente
5	Prueba 4	Alta	Entornos	5 may 2026	15 ago 2026	pendiente
4	Prueba 3	Media	Programación	5 may 2026	21 jun 2026	pendiente
2	Prueba 1	Baja	Programación	5 may 2026	6 may 2026	pendiente

Ordenado por fecha

Tareas Pendientes

Ordenar por Prioridad

Ordenar por Modulo

Ordenar por Fecha de entrega

Código	Título	Prioridad	Módulo	Fecha creación	Fecha entrega	Estado
3	Prueba 2	Alta	Bases de datos	5 may 2026	19 ene 2026	pendiente
2	Prueba 1	Baja	Programación	5 may 2026	6 may 2026	pendiente
4	Prueba 3	Media	Programación	5 may 2026	21 jun 2026	pendiente
5	Prueba 4	Alta	Entornos	5 may 2026	15 ago 2026	pendiente